

ORGANIZACIÓN Y ESTRUCTURA DE DATOS

Unidad 3

Organización de Archivos Directos y Técnicas de Hashing

Material de Estudio — Universidad de Ingeniería Informática

2025

Índice de Contenidos

1. Organización de Archivos Directos — Estructura Conceptual
2. ¿Qué es el Acceso Directo?
3. Fundamentos y Principios del Acceso Directo
4. Características de la Organización Directa
5. Requisitos para el Acceso Directo Eficiente
6. Desafíos y Soluciones — Introducción al Hashing
7. Comparativa de Métodos de Acceso
8. Definición Formal y TAD Hashing
9. Métricas de Rendimiento: Factor de Carga y Densidad
10. Técnicas de Construcción de Funciones Hash
 - 10.1 Resto de División (Módulo)
 - 10.2 Multiplicación (Knuth)
 - 10.3 Plegado — Fold & Add
 - 10.4 Compresión XOR
 - 10.5 Extracción del Cuadrado Medio
 - 10.6 Funciones de Resumen (SHA-256)
11. Colisiones: Concepto y Estrategias de Resolución
 - 11.1 Progressive Overflow (Sondeo Lineal)
 - 11.2 Doble Hashing
 - 11.3 Almacenamiento por Cubetas
 - 11.4 Tablas Dispersas (Scatter Tables)
 - 11.5 Encadenado con Área de Overflow Separada
12. Hash Extendido
13. Conclusiones y Comparativa Final

1. Organización de Archivos Directos — Estructura Conceptual

La **organización de archivos directos** es una técnica fundamental en informática que permite acceder a registros almacenados en memoria secundaria sin necesidad de recorrer todos los elementos anteriores. Esta clase de organización se contrapone al acceso secuencial, en el que la búsqueda de un dato exige revisar registro por registro desde el inicio.

Desde el punto de vista estructural, los archivos pueden ser binarios o de texto y contienen registros que pueden ser de **longitud fija** o de **longitud variable**. Esta distinción es crítica: solo los registros de longitud fija admiten acceso directo, porque solo en ese caso es posible calcular matemáticamente la posición exacta de cualquier registro a partir de su número o clave.

Las técnicas que hacen posible la organización directa incluyen: búsqueda binaria, ordenamiento Shell, dispersión Hash y árboles. Cada una ofrece distintas garantías de rendimiento. El Hashing, en particular, es la técnica estrella de esta unidad porque aspira al acceso en tiempo constante $O(1)$.

Tipo de acceso	Estructura requerida	Ejemplo de técnica
Secuencial	Longitud variable o fija	Recorrido lineal
Directo	Longitud fija, clave primaria	Hashing, Indexamiento

2. ¿Qué es el Acceso Directo?

El **acceso directo** es la capacidad de localizar un registro de manera inmediata a partir de su clave, sin necesidad de examinar los registros que lo preceden. La idea central es que existe una función que transforma la clave del registro en una dirección física de almacenamiento, y esa transformación puede ejecutarse en tiempo constante.

Un aspecto clave que suele generar confusión es que la **secuencia lógica** de los registros no necesariamente coincide con la **secuencia física** en el dispositivo de almacenamiento. Es decir, el registro número 5 lógicamente puede estar físicamente en la posición 847 del disco. Esto no es un problema: la función de transformación se encarga de mapear de un espacio al otro.

■ Ventaja principal

El acceso directo tiene como caso ideal la complejidad $O(1)$, es decir, el tiempo de búsqueda no crece con la cantidad de registros. Esto lo hace enormemente superior al acceso secuencial para archivos grandes.

3. Fundamentos y Principios del Acceso Directo

El acceso directo se estructura sobre tres pasos fundamentales que ocurren cada vez que se quiere leer o escribir un registro:

Paso 1 — Identificación de la Clave: Cada registro tiene un campo (o combinación de campos) denominado **clave primaria** que lo identifica de manera única dentro del archivo. Puede ser un número de DNI, un código de producto, un número de legajo, etc.

Paso 2 — Transformación: Se aplica una función matemática que convierte esa clave en un número de registro relativo (NRR), es decir, en una dirección física dentro del archivo. Esta función se llama *función de hashing* o *función de dispersión*.

Paso 3 — Acceso Directo: Con la dirección calculada, el sistema operativo puede ir directamente a esa posición del archivo y leer o escribir el registro. No es necesario leer ningún otro registro en el camino.

■ Definición formal de Clave Primaria	Campo o combinación de campos que identifican de manera única cada registro del archivo. Puede ser simple (un solo atributo) o compuesta (varios atributos que juntos son únicos).
--	--

4. Características de la Organización Directa

4.1 Medios de Almacenamiento Adecuados

No todos los dispositivos de almacenamiento son aptos para la organización directa. La **memoria principal (RAM)** y los **discos magnéticos** (HDD, SSD) permiten acceder a cualquier posición en tiempo similar, lo que hace viable el acceso directo. Las **cintas magnéticas**, en cambio, son inherentemente secuenciales: para llegar a la posición N hay que pasar por todas las anteriores, lo que invalida cualquier ventaja del acceso directo.

4.2 Huecos en el Espacio de Almacenamiento

Una consecuencia natural del acceso directo es que el archivo puede tener **posiciones vacías** entre registros válidos. Si las claves no son densas (es decir, si hay muchos valores posibles que no están usados), existirán huecos. La tabla siguiente ilustra esto: las posiciones 2, 3 y N están marcadas como 'nulo', lo que significa que ese espacio está reservado pero no contiene un registro válido.

Posición Física	Clave almacenada
0	20438705
1	40436878
2	nulo (vacío)
3	nulo (vacío)

4	48878956
5	54567896
N	nulo (vacío)

4.3 Complejidad de Programación

La organización directa requiere que el programador establezca y mantenga la relación entre el contenido lógico del registro y su posición física. Esto añade complejidad respecto al acceso secuencial, pero la compensación es la eficiencia: una vez implementada correctamente, cada búsqueda tarda lo mismo independientemente del tamaño del archivo.

5. Requisitos para el Acceso Directo Eficiente

Para que la organización directa funcione correctamente y sin desperdiciar recursos, deben cumplirse cuatro condiciones:

- 1. Orden ascendente de claves:** El conjunto de claves debe mantener un orden ascendente con pocos valores intermedios no utilizados. Si hay muchos huecos entre valores consecutivos, el desperdicio de espacio en disco se vuelve inaceptable.
- 2. Correspondencia ideal:** En el caso ideal, cada clave coincide directamente con un número de registro relativo. Por ejemplo, el empleado con legajo 542 se almacena en la posición 542 del archivo. Cuando esto no es posible (claves alfanuméricas, DNI, etc.), se necesita una función de transformación.
- 3. Unicidad de direcciones:** Debe existir exactamente una dirección de almacenamiento por cada valor posible de clave. Dos claves distintas no pueden apuntar a la misma dirección si cada posición almacena un único registro (sin el uso de cubetas).
- 4. Rango acotado:** Los valores de las claves deben pertenecer a un rango limitado y predecible. Si el rango es impredeciblemente grande, el tamaño del archivo crecería descontroladamente para acomodar todos los posibles valores.

6. Desafíos y Soluciones — Introducción al Hashing

En la práctica, las claves casi nunca corresponden directamente a direcciones físicas. Un DNI tiene 8 dígitos (valores de 0 a 99.999.999), pero un archivo de empleados puede tener solo 23.000 registros. Reservar 100 millones de posiciones sería absurdo. Aquí entra el **Hashing**.

El Hashing (dispersión) es una técnica que transforma una clave de cualquier tipo en una dirección dentro de un rango manejable. La función de transformación, llamada **función hash**, aplica operaciones matemáticas a la clave y produce un número de dirección. El proceso es tan rápido que se ejecuta en tiempo constante, y no requiere almacenar un índice auxiliar.

Las características técnicas de un archivo con organización directa basada en Hashing son:

- El archivo tiene **longitud fija** y generalmente no es extensible.
- Los registros tienen **longitud fija** con una clave primaria identificable.
- Cada posición tiene un **atributo de estado**: libre, ocupado o borrado.
- Se usa una **función de hashing** para calcular la posición de cada registro.

■ Origen histórico

El término 'hash' proviene del inglés: picar y mezclar. Donald Knuth atribuye su primera utilización a H.P. Luhn de IBM en enero de 1953, aunque su uso masivo no comenzó hasta una década después.

Los tres desafíos principales al implementar Hashing son:

Hashing: Diseñar una función que distribuya las claves uniformemente.

Granularidad: Decidir si cada posición almacena un registro o un bloque de registros (cubeta).

Overflow (desbordamiento): Manejar el caso en que la posición calculada ya está ocupada.

7. Comparativa de Métodos de Acceso

Para un archivo de 10.000 registros, el número de accesos a disco necesarios para encontrar un registro varía drásticamente según el método de organización empleado:

Método	Accesos necesarios	Condición
Secuencial ordenado	~13 accesos	$\log_{10}(10.000) \approx 13$
Árboles balanceados	< 3 accesos	Orden d=16, factor de carga 3/4
Hashing ideal	1 acceso	Sin colisiones, O(1)

La diferencia es enorme: con 10.000 registros, el acceso secuencial necesita en promedio 13 veces más trabajo que el Hashing ideal. Con 1.000.000 de registros, serían aproximadamente 20 accesos para búsqueda binaria frente a 1 para Hashing. La pregunta clave es: ¿se puede diseñar una función que siempre mapee claves distintas a direcciones distintas? En general, la respuesta es no — y ahí surge el problema de las **colisiones**.

8. Definición Formal y TAD Hashing

8.1 Definición Matemática

Formalmente, una función de hashing se define como una aplicación que va del conjunto de claves posibles al conjunto de direcciones de memoria:

Función de Transformación H

$H: X \rightarrow D$

donde:

X = conjunto de claves posibles (dominio)

D = conjunto de direcciones (rango)

$H(x)$ = función de transformación

Es una aplicación suprayectiva: todo elemento de D tiene al menos un elemento de X que lo genera (pero no necesariamente al revés).

La función H transforma claves de cualquier naturaleza (números, cadenas de texto, emails, etc.) en un número entero que representa una posición dentro del archivo. Cuando las claves son textuales, primero se convierten a su representación numérica (por ejemplo, mediante los valores ASCII de sus caracteres) y luego se aplica la función.

8.2 Operaciones del TAD TablaHash

Las tablas hash constituyen un Tipo Abstracto de Datos (TAD) con las siguientes operaciones:

Operación	Efecto	Excepción
Inicializar()	Crea una tabla hash vacía.	—
Hash(clave) $\rightarrow$ posición	Calcula y devuelve la dirección correspondiente.	—
Buscar(tabla, clave) $\rightarrow$ atributo	Devuelve el registro si existe; error=false.	error=true si no existe
Insertar(tabla, clave, atrib)	Almacena el par (clave, atributo) en Hash(clave).	error si colisión/tabla llena
Eliminar(tabla, clave)	Marca como libre la posición del registro.	error si no existe

■ Advertencia

Eliminar un registro NO borra físicamente el contenido: solo se cambia el atributo de estado de 'ocupado' a 'borrado'. Esto es crítico para que los algoritmos de búsqueda funcionen correctamente en presencia de colisiones resueltas por sondeo.

8.3 Colisión y Desbordamiento

Una **colisión** ocurre cuando dos claves distintas producen la misma dirección hash: $H(k_1) = H(k_2)$ con $k_1 \neq k_2$. El **desbordamiento** ocurre cuando se intenta insertar en una posición (o cubeta) que ya está completamente ocupada. En el caso particular de que cada posición almacene exactamente un registro, colisión y desbordamiento son el mismo evento.

Las funciones de hashing se clasifican en **perfectas** (sin colisiones, posible solo cuando se conoce de antemano el conjunto completo de claves) e **imperfectas** (pueden producir colisiones, que es el caso general en la práctica). En producción siempre se trabaja con funciones imperfectas y se diseñan estrategias de resolución de colisiones.

9. Métricas de Rendimiento: Factor de Carga y Densidad

Dos métricas cuantitativas permiten evaluar cómo se comportará una tabla hash antes de implementarla, y son esenciales para tomar decisiones de diseño.

9.1 Densidad de Claves

La densidad de claves mide qué tan ocupado está el universo de claves posibles respecto al espacio total que ese universo define. Si la densidad es muy baja, significa que las claves usadas son una fracción pequeña de las posibles, y el acceso directo puro desperdiciaría mucha memoria; en ese caso conviene usar Hashing.

Densidad de claves

$$\delta_{\text{claves}} = |K| / |X| = R / |X|$$

donde:

$|X|$ = tamaño del universo de claves posibles

$R = |K|$ = cantidad de claves efectivamente usadas

9.2 Factor de Carga (α)

El factor de carga mide qué tan llena está la tabla hash. Es la métrica más usada en la práctica porque predice directamente la probabilidad de colisiones: a mayor factor de carga, más colisiones se esperan.

Factor de carga

$$\alpha = R / N$$

Con cubetas de capacidad C:

$$\alpha_{ef} = R / (N \times C)$$

donde:

R = número de registros insertados

N = número de direcciones/buckets disponibles

C = capacidad de registros por cubeta

9.3 Distribución de Poisson y Probabilidad de Colisiones

Como la función hash distribuye claves de manera aproximadamente aleatoria, la probabilidad de que exactamente x registros caigan en la misma dirección sigue una **distribución de Poisson**:

Fórmula de Poisson aplicada al Hashing

$$p(x) = (R/N)^x \times e^{-(R/N)} / x!$$

donde:

p(x) = probabilidad de que exactamente x registros caigan en la misma dirección

R/N = factor de carga (λ de la distribución de Poisson)

Ejemplo numérico: N = 1000 buckets, R = 1000 registros → $\alpha = 1$

P(2 registros en la misma dirección) = 0.184 (18.4%)

P(3 registros en la misma dirección) = 0.061 (6.1%)

P(4 registros en la misma dirección) = 0.015 (1.5%)

→ Aunque en promedio cada registro ocupa una posición única, casi el 18% de las posiciones tendrán colisiones con al menos otro registro.

Ejemplo de densidad: 75 registros en 100 buckets → densidad = 0.75

→ A mayor densidad, más colisiones. Se recomienda $\alpha < 0.75$ en la práctica.

10. Técnicas de Construcción de Funciones Hash

Una buena función hash debe distribuir las claves lo más uniformemente posible entre todas las posiciones disponibles, ser rápida de computar y ser determinista (la misma clave siempre produce la misma dirección). A continuación se describen las técnicas principales.

10.1 Resto de División (Módulo)

Es la técnica más simple y una de las más usadas. La clave se divide entre el tamaño de la tabla y se toma el resto como dirección. Su eficiencia depende fuertemente de la elección del divisor m .

Fórmula del método módulo

$$h(k) = k \bmod m$$

donde:

k = clave (entera o convertida a entero)

m = tamaño de la tabla (conviene que sea número primo)

Ejemplo: $k = 123456$, $m = 1009$ (número primo)

$$h(123456) = 123456 \bmod 1009 = 358$$

$$h(987654321) = 987654321 \bmod 1009 = 725$$

Ejemplo simple con $m = 10$:

$$k=18 \rightarrow h=8 \quad | \quad k=23 \rightarrow h=3 \quad | \quad k=36 \rightarrow h=6$$

$$k=41 \rightarrow h=1 \quad | \quad k=57 \rightarrow h=7$$

■ Advertencia

Evitar usar potencias de 2 como valor de m . Si $m = 2^p$, entonces $h(k)$ equivale a los p bits de menor orden de k , ignorando toda la información de los bits superiores. Se recomienda un número primo no cercano a potencias de 2 ni múltiplos de 10.

10.2 Multiplicación (Método de Knuth)

El método de multiplicación es menos sensible a los patrones de la clave y funciona bien incluso cuando m no es primo. Se basa en multiplicar la clave por una constante irracional A , extraer la parte fraccionaria del resultado y escalarla al tamaño de la tabla.

Fórmula del método de multiplicación

$$h(k) = \lfloor m \times \{kA\} \rfloor$$

donde:

$\{x\}$ = parte fraccionaria de x ($x - \lfloor x \rfloor$)

$A \in (0,1)$, constante irracional recomendada:

$A = (\sqrt{5} - 1) / 2 \approx 0.6180339887$ (razón áurea)

m = tamaño de la tabla (puede ser potencia de 2)

Pasos prácticos para $k = 18$, $m = 10$, $A = 0.6180339887$:

1. Calcular: $18 \times 0.6180339887 = 11.1246$

2. Parte fraccionaria: $11.1246 - 11 = 0.1246$

3. Escalar: $\lfloor 10 \times 0.1246 \rfloor = \lfloor 1.246 \rfloor = 1 \rightarrow h(18) = 1$

Para $k = 23$: $23 \times A = 14.2148 \rightarrow \{14.2148\} = 0.2148 \rightarrow h = 2$

Para $k = 36$: $36 \times A = 22.2492 \rightarrow \{22.2492\} = 0.2492 \rightarrow h = 2$

(colisión entre $k=23$ y $k=36$)

Ejemplo con $m=1024$:

$h(123456) = 4 \mid h(987654321) = 526$

10.3 Plegado — Fold & Add

El plegado es útil cuando las claves son largas (muchos dígitos o caracteres). Se divide la clave en bloques de igual tamaño y se suman todos los bloques. El resultado, reducido mediante módulo, da la dirección. Existen dos variantes:

Variante A — Plegado por desplazamiento (shift folding): Se suman los bloques tal como están, sin ninguna modificación.

Variante B — Plegado por fronteras (boundary folding): Los bloques en posiciones pares se invierten (se leen de derecha a izquierda) antes de sumarlos. Esto mezcla mejor los bits y reduce las colisiones cuando las claves tienen patrones repetitivos.

Fórmula general del plegado

$$h(k) = (\sum B_i) \bmod m \text{ [shift folding]}$$

$$h(k) = (\sum f_i(B_i)) \bmod m \text{ [boundary folding, } f_i \text{ invierte los bloques pares]}$$

donde B_i son los bloques numéricos en que se divide la clave k

Ejemplo: k = 12345678, bloques de 2 dígitos, m = 100

Bloques: [12, 34, 56, 78]

Shift folding: $12 + 34 + 56 + 78 = 180 \rightarrow 180 \bmod 100 = 80$

Boundary folding: invertimos los pares $\rightarrow [12, 43, 56, 87]$

$$12 + 43 + 56 + 87 = 198 \rightarrow 198 \bmod 100 = 98$$

Ejemplo con clave textual 'GIDI' (valores ASCII: G=71, I=73, D=68, I=73), m=128:

Shift: $71+73+68+73 = 285 \rightarrow 285 \bmod 128 = 29$

Boundary decimal: $71+37+68+37 = 213 \rightarrow 213 \bmod 128 = 85$

Boundary binario (bit-reverse): $71+146+68+146 = 431 \rightarrow 431 \bmod 128 = 47$

Ejemplo con clave alfanumérica: 'pepe@gmail.com', tabla de 10.000 posiciones

ASCII: 112 101 112 101 64 103 109 97 105 108 46 99 111 109

Bloques de 4 dígitos: $1121 + 0111 + 2101 + 6410 + 3109 + 9710 + 5108 + \dots = 34380$

$$\text{hash} = 34380 \bmod 10007 = 4359$$

10.4 Compresión — Hash por XOR

La compresión es especialmente útil para claves textuales. Se convierte cada carácter a su representación binaria y se aplica la operación XOR (OR exclusivo) bit a bit sobre todos los caracteres. El resultado se reduce con módulo.

Ejemplo: $k = \text{'David'}$, $m = 26$

D = 01100100

A = 01100001

V = 01110010 ← XOR acumulado

I = 01101001

D = 01100100

[illegible]

XOR = 01111010 = 122 (decimal)

$$h('David') = 122 \bmod 26 = 18$$

10.5 Extracción del Cuadrado Medio

Este método eleva la clave al cuadrado y extrae un subconjunto de dígitos de la zona central del resultado. La justificación es que los dígitos del centro del cuadrado dependen de todos los dígitos de la clave original, lo que produce una mejor mezcla.

Ejemplo: tabla m = 100 (2 dígitos), tomar posiciones centrales del cuadrado

K=122: $122^2 = 014884 \rightarrow$ dígitos centrales (pos 3-4) = 48 $\rightarrow$ índice 48

K=341: $341^2 = 116281 \rightarrow$ dígitos centrales = 62 $\rightarrow$ índice 62

K=525: $525^2 = 275625 \rightarrow$ dígitos centrales = 56 $\rightarrow$ índice 56

K=612: $612^2 = 374544 \rightarrow$ dígitos centrales = 45 $\rightarrow$ índice 45

Ejemplo con DNI=35678789, tabla de 10.000 posiciones (4 dígitos):

$K^2 = 35678789^2 = 1272913441351121$ (16 dígitos)

Inicio del centro: $(16/2) - (4/2) = 8-2 = 6 \rightarrow$ posiciones 7 a 10

hash_valor = 3441

10.6 Funciones de Resumen Criptográfico (SHA-256)

Cuando se necesita la máxima calidad de distribución y seguridad, se puede usar una función de resumen criptográfico como SHA-256. El hash de 256 bits resultante se convierte a número entero y se reduce con módulo. Es computacionalmente más costoso, pero prácticamente elimina los patrones de colisión.

Ejemplo: k = 'pepe@gmail.com', tabla de 10.000 posiciones

`sha256('pepe@gmail.com')` = 70b6a4bd4293c44a7ebbf692aacee7...

Primeros 8 caracteres hex $\rightarrow$ entero: 704155382

hash = $704155382 \bmod 10000 = 5382$

■
**Aplicaciones
del Hashing
más allá de
archivos**

Las funciones hash unidireccionales (como MD5 de 128 bits y SHA de 160+ bits) tienen aplicaciones en: verificación de integridad de documentos, firmas digitales, almacenamiento seguro de contraseñas (con salt) y sellos de tiempo.

11. Colisiones: Concepto y Estrategias de Resolución

Una colisión es inevitable en la práctica: ocurre cuando $H(k_1) = H(k_2)$ con $k_1 \neq k_2$. El **algoritmo de hash perfecto** (sin colisiones) solo existe cuando se conoce de antemano el conjunto exacto de claves. En el caso general hay que diseñar estrategias para manejarlas. Las soluciones se agrupan en dos grandes categorías:

Categoría		Idea central	Ejemplo de técnica
Direccionamiento	abierto	El elemento se ubica en otra celda de la misma tabla.	Progressive Overflow, Doble Hashing
Encadenamiento	(hashing abierto)	Cada posición contiene una lista de elementos.	Cubetas con listas, Scatter Tables

■ Advertencia

Nota terminológica: en la literatura se usa 'hashing cerrado' para referirse a que todo está dentro de la tabla (aunque el sondeo esté 'abierto'), y 'hashing abierto' para el encadenamiento que usa estructuras externas. Los nombres son contraintuitivos — es importante no confundirlos.

11.1 Progressive Overflow (Sondeo Lineal)

Es el método de resolución de colisiones más popular por su simplicidad y eficiencia. Cuando una inserción encuentra la posición calculada ocupada, se avanza de forma secuencial por la tabla hasta encontrar una posición libre.

Secuencia de sondeo lineal

$$P_i = (h(k) + i) \bmod m, \quad i = 0, 1, 2, \dots$$

→ Si la tabla se llena al final, el sondeo vuelve al inicio (circularmente).

→ Una posición 'libre' puede ser vacía o marcada como 'borrada'.

Ejemplo: $m=7$, $h(k) = k \bmod 7$, inserción de: 18, 41, 22, 44, 59, 32, 31

$k=18$: $h=4 \rightarrow$ posición 4 libre $\rightarrow$ coloca en 4

$k=41$: $h=6 \rightarrow$ posición 6 libre $\rightarrow$ coloca en 6

$k=22$: $h=1 \rightarrow$ posición 1 libre $\rightarrow$ coloca en 1

$k=44$: $h=2 \rightarrow$ posición 2 libre $\rightarrow$ coloca en 2

$k=59$: $h=3 \rightarrow$ posición 3 libre $\rightarrow$ coloca en 3

$k=32$: $h=4 \rightarrow$ pos 4 ocupada $\rightarrow$ sondeo $\rightarrow$ pos 5 libre $\rightarrow$ coloca en 5

$k=31$: $h=3 \rightarrow$ pos 3 ocup $\rightarrow$ pos 4 ocup $\rightarrow$ pos 5 ocup $\rightarrow$ pos 6 ocup $\rightarrow$ pos 0 libre $\rightarrow$ coloca en 0

Resultado final: [31, 22, 44, 59, 18, 32, 41] (posiciones 0..6)

■ Advertencia

El sondeo lineal sufre de 'agrupamiento primario' (primary clustering): los registros tienden a agruparse en zonas de la tabla, lo que ralentiza las búsquedas. Si el factor de carga supera 0.7-0.8, el rendimiento se degrada significativamente.

11.2 Doble Hashing

El doble hashing usa una segunda función hash para calcular el salto entre intentos, en lugar del salto fijo 1 del sondeo lineal. Esto elimina el agrupamiento primario y produce una distribución más uniforme.

Doble hashing

$$P_i = (h_1(k) + i \times h_2(k)) \bmod m, \quad i = 0, 1, 2, \dots$$

Donde $h_2(k)$ es el paso, que depende de la clave.

Fórmula recomendada para h_2 :

$$h_2(k) = 1 + (k \bmod (m-1)) \rightarrow \text{garantiza } 1 \leq h_2 \leq m-1$$

Condición: $\gcd(h_2(k), m) = 1 \rightarrow$ visita todas las celdas

Ejemplo: $m=7$, $h_1(k)=k \bmod 7$, $h_2(k)=1+(k \bmod 6)$, mismas claves que antes

$k=18$: $h_1=4$, $h_2=1 \rightarrow$ sondeo: 4 $\rightarrow$ coloca en 4

$k=41$: $h_1=6$, $h_2=6 \rightarrow$ sondeo: 6 $\rightarrow$ coloca en 6

$k=22$: $h_1=1$, $h_2=5 \rightarrow$ sondeo: 1 $\rightarrow$ coloca en 1

$k=44$: $h_1=2$, $h_2=3 \rightarrow$ sondeo: 2 $\rightarrow$ coloca en 2

$k=59$: $h_1=3$, $h_2=6 \rightarrow$ sondeo: 3 $\rightarrow$ coloca en 3

$k=32$: $h_1=4$, $h_2=3 \rightarrow$ pos 4 ocup $\rightarrow (4+3) \bmod 7=0 \rightarrow$ coloca en 0

$k=31$: $h_1=3$, $h_2=2 \rightarrow$ pos 3 ocup $\rightarrow (3+2) \bmod 7=5 \rightarrow$ coloca en 5

Resultado: [32, 22, 44, 59, 18, 31, 41]

$\rightarrow$ Mejor distribución que el sondeo lineal (sin agrupamiento).

11.3 Almacenamiento por Cubetas (Buckets)

En lugar de asignar una celda individual por registro, se asignan **cubetas** (bloques) capaces de almacenar C registros cada una. Varias claves que producen el mismo hash conviven dentro de la misma cubeta sin necesidad de sondeo adicional. Solo cuando una cubeta se llena completamente se activa el mecanismo de overflow.

El factor de carga efectivo con cubetas es $\alpha_{ef} = R / (N \times C)$. Si $C=4$ y $\alpha_{ef} < 0.75$, la probabilidad de que una cubeta se desborde es muy baja.

Ejemplo: $N=5$ cubetas (0..4), $h(k)$ = últimos 2 dígitos del teléfono mod 5

Ana López 555-1012 $\rightarrow 12 \bmod 5 = 2 \rightarrow$ Bucket 2
 Luis López 555-9917 $\rightarrow 17 \bmod 5 = 2 \rightarrow$ Bucket 2 (colisión, misma cubeta OK)
 Marta Díaz 555-0032 $\rightarrow 32 \bmod 5 = 2 \rightarrow$ Bucket 2 (otra colisión)
 Pedro Ruiz 555-2044 $\rightarrow 44 \bmod 5 = 4 \rightarrow$ Bucket 4
 Sofía Ruiz 555-2049 $\rightarrow 49 \bmod 5 = 4 \rightarrow$ Bucket 4 (colisión)
 Tomás Vega 555-0101 $\rightarrow 01 \bmod 5 = 1 \rightarrow$ Bucket 1
 Carla Pérez 555-1005 $\rightarrow 05 \bmod 5 = 0 \rightarrow$ Bucket 0
 Bruno Álvarez 555-2016 $\rightarrow 16 \bmod 5 = 1 \rightarrow$ Bucket 1 (colisión)

Resultado:

Bucket 0: [Carla Pérez]
 Bucket 1: [Tomás Vega $\rightarrow$ Bruno Álvarez]
 Bucket 2: [Ana López $\rightarrow$ Luis López $\rightarrow$ Marta Díaz]
 Bucket 3: [vacío]
 Bucket 4: [Pedro Ruiz $\rightarrow$ Sofía Ruiz]

11.4 Tablas Dispersas — Scatter Tables

En las tablas dispersas no existe un área primaria de datos con posiciones contiguas. En cambio, cada posición de la tabla contiene un puntero a una lista enlazada donde residen los registros sinónimos (los que colisionan en esa dirección). Los nodos de la lista pueden estar físicamente dispersos en el almacenamiento.

Las ventajas son la flexibilidad y la facilidad para borrar registros sin complicar las búsquedas. Las desventajas son la menor localidad de referencia (los punteros impiden aprovechar el caché) y la mayor complejidad de implementación. Para acceso en disco, cada salto de puntero puede implicar una operación de E/S adicional.

11.5 Encadenado con Área de Overflow Separada

Esta técnica combina un área primaria donde se almacena el primer registro de cada dirección con un **área de overflow separada** donde se colocan los registros adicionales que colisionan. El área primaria solo guarda la cabeza (puntero) de la lista; los demás elementos viven en el área de overflow y se encadenan mediante punteros.

La búsqueda es: ir a la dirección hogar en el área primaria, leer el puntero y seguir la cadena en el área de overflow hasta encontrar la clave o llegar a un puntero nulo (-1).

12. Hash Extendido

Uno de los problemas fundamentales del Hashing clásico es que el archivo tiene tamaño fijo: si crece demasiado, hay que rehashear todo, lo que es una operación costosa. El **Hash Extendido** soluciona esto permitiendo que el archivo crezca dinámicamente sin cambiar la función de hashing.

Para lograrlo, se usan cuatro estructuras interdependientes:

- 1. Archivo de tabla de dispersión** (directorío): Almacena punteros a bloques de datos y un valor llamado profundidad global que determina cuántos bits de la clave hash se usan para indexar la tabla.
- 2. Archivo de bloques** (páginas de datos): Cada bloque almacena una cantidad fija de registros (longitud fija) y tiene su propia profundidad local. Múltiples entradas del directorío pueden apuntar al mismo bloque.
- 3. Lista de bloques libres**: Gestiona el espacio disponible para nuevos bloques cuando un bloque existente se desborda y debe dividirse.
- 4. Función de hashing**: Se aplica a la clave para obtener un valor binario; luego se usan los primeros d bits (profundidad global) para indexar el directorío.

■ Mecanismo de crecimiento

Cuando un bloque se llena, se divide en dos y se redistribuyen sus registros. Si la profundidad local del bloque era igual a la global, el directorío también se duplica. Este proceso es local y no requiere reorganizar todo el archivo.

■ Advertencia

Limitaciones del Hash Extendido: aunque crece dinámicamente (como los B-Trees), no es óptimo para acceso secuencial ordenado, donde el B+Tree sigue siendo superior. Además, carece de algunas métricas importantes sobre la capacidad de cada cubeta.

13. Conclusiones y Comparativa Final

La organización directa de archivos mediante Hashing representa el paradigma de mayor velocidad de acceso en la gestión de datos. En el caso ideal, toda búsqueda, inserción y eliminación se ejecuta en tiempo $O(1)$, independientemente del tamaño del archivo. Esto la convierte en la base de estructuras de datos críticas como las tablas hash de los lenguajes de programación modernos, los índices de bases de datos y los sistemas de caché.

Aspecto	Hashing	Árboles (B-Tree)	Secuencial
Búsqueda	$O(1)$ ideal	$O(\log n)$	$O(n)$ / $O(\log n)$ binaria
Inserción	$O(1)$ amortizado	$O(\log n)$	$O(n)$

Acceso ordenado	No soportado	Eficiente (B+Tree)	Eficiente
Crecimiento dinámico	Hash Extendido	Sí (natural)	Limitado
Colisiones	Requieren manejo	No aplica	No aplica

Las decisiones de diseño más importantes al implementar Hashing son: (1) elegir una buena función hash que distribuya uniformemente, (2) dimensionar correctamente la tabla para mantener el factor de carga bajo (idealmente $\alpha < 0.75$), y (3) seleccionar la estrategia de resolución de colisiones adecuada al caso de uso: sondeo lineal para tablas pequeñas y acceso en memoria, encadenamiento para tablas grandes en disco donde el overhead de punteros es aceptable.

En resumen, el Hashing no es una solución única ni perfecta, pero es la técnica más eficiente para el acceso puntual a registros cuando se conoce la clave. Su comprensión profunda, incluyendo sus limitaciones, es esencial para el diseño de sistemas de información de alto rendimiento.

Material de estudio elaborado sobre la base de la presentación de la Cátedra — Organización y Estructura de Datos — Ingeniería Informática — 2025.